PBJ-2 Practical Assignment

Lab 2: End-to-End E-Commerce Order Processing and Fulfilment

Name:Sunandha Kumar
Register No.: RA2411003010157

1. Objective

Build an order-processing workflow on Camunda 8 that validates an order, checks simulated inventory, determines delivery priority with DMN, processes a simulated payment through a REST Connector, and routes each order to fulfilment, rejection or human verification. Operate is used to monitor instances and recover from an inventory-service failure. All customer, product and payment data is synthetic; payment tokens are fake strings for a mock gateway.

2. Environment and Tools

Component	Version / details
Camunda 8 Run (c8run)	8.8.32, local: Zeebe, Operate, Tasklist, Connectors runtime
Camunda Desktop Modeler	5.49.0, target platform Camunda 8.8
Java / Spring Boot	OpenJDK 21, Spring Boot 3.5.16, camunda-spring-boot-starter 8.8.32
IDE / build	IntelliJ IDEA, Maven
Mock services	Payment gateway and inventory outage switch in the same app, port 8091

3. Architecture

- **Camunda 8 Run** executes the process, evaluates the DMN, runs the REST Connector and serves Tasklist and Operate.
- **Spring Boot application** (port 8091) hosts the `validate-order` job worker, a simulated inventory (`InventoryService`) and the mock payment gateway `POST /api/payments`.

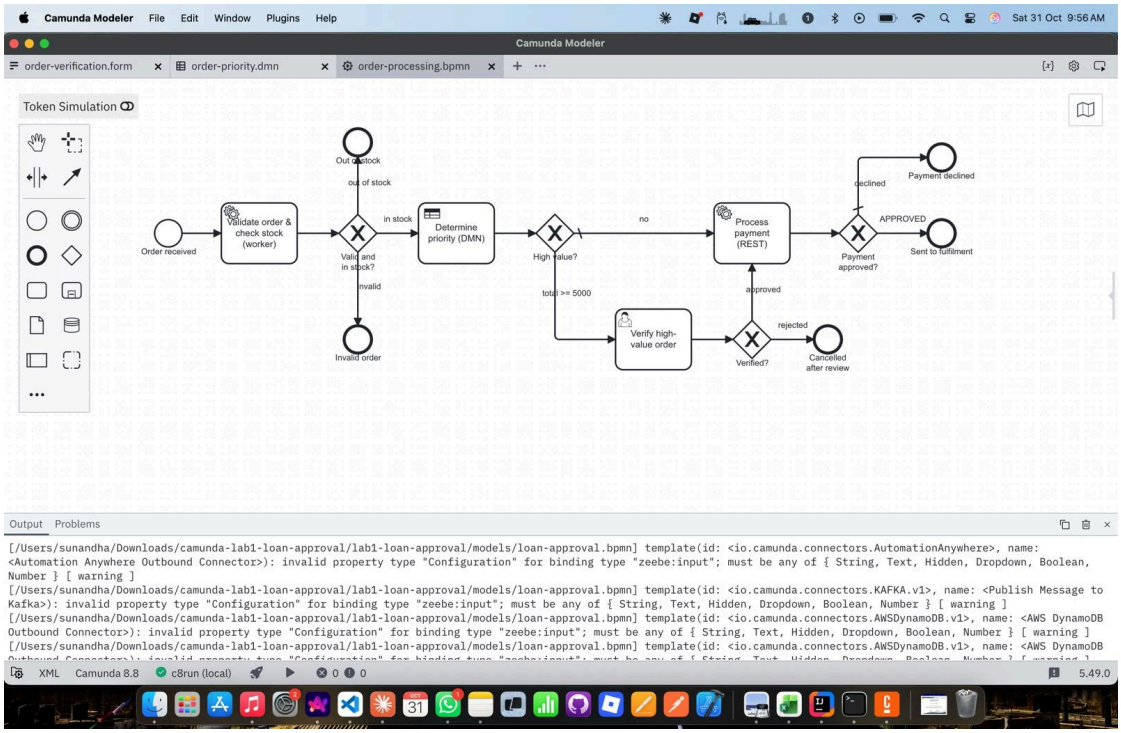


Figure 1: *Order Processing BPMN process in Camunda Desktop Modeler (process id order-processing)*

Execution flow: Order received → Validate order & check stock (Java worker) → Valid and in stock? (invalid → Invalid order; out of stock → Out of stock) → Determine priority (DMN) → High value? (orderTotal >= 5000 → Verify high-value order in Tasklist → approved continues / rejected → Cancelled after review) → Process payment (REST Connector) → Payment approved? → Sent to fulfilment or Payment declined.

4. Implementation

4.1 BPMN process

Element	Type	Configuration
Validate order & check stock	Service task	Job type `validate-order`, 3 retries (Java worker)
Valid and in stock?	Exclusive gateway	Invalid / out of stock / in stock (see 4.6)
Determine priority	Business rule task	DMN `order-priority`, result variable `deliveryPriority`
High value?	Exclusive gateway	`orderTotal >= 5000` → verification; default → payment
Verify high-value order	User task (Camunda user task)	Assignee `demo`, linked form `order-verification`
Verified?	Exclusive gateway	`verificationDecision` = APPROVED / REJECTED
Process payment	Service task (REST Connector)	`io.camunda:http-json:1`, POST mock gateway, 3 retries
Payment approved?	Exclusive gateway	`paymentStatus = "APPROVED"`; default → Payment declined

4.2 DMN priority decision table

Decision **`order-priority`**, hit policy **FIRST**. Inputs: **`deliveryPreference`**, **`customerCategory`**, **`orderTotal`**. Output: **`deliveryPriority`**.

Rule	deliveryPreference	customerCategory	orderTotal	deliveryPriority
R1	"EXPRESS"	–	–	HIGH
R2	–	"GOLD"	>= 1000	HIGH
R3	–	"GOLD"	–	MEDIUM
R4	–	–	>= 1000	MEDIUM
R5	–	–	–	LOW

Rationale: express delivery always needs fast handling; gold customers are favoured; large orders get at least medium priority. Sample evaluations from the runs: C-100 (STANDARD, REGULAR, 50) → R5 LOW; C-300 (STANDARD, GOLD, 100) → R3 MEDIUM; C-400 and C-500 (EXPRESS) → R1 HIGH.

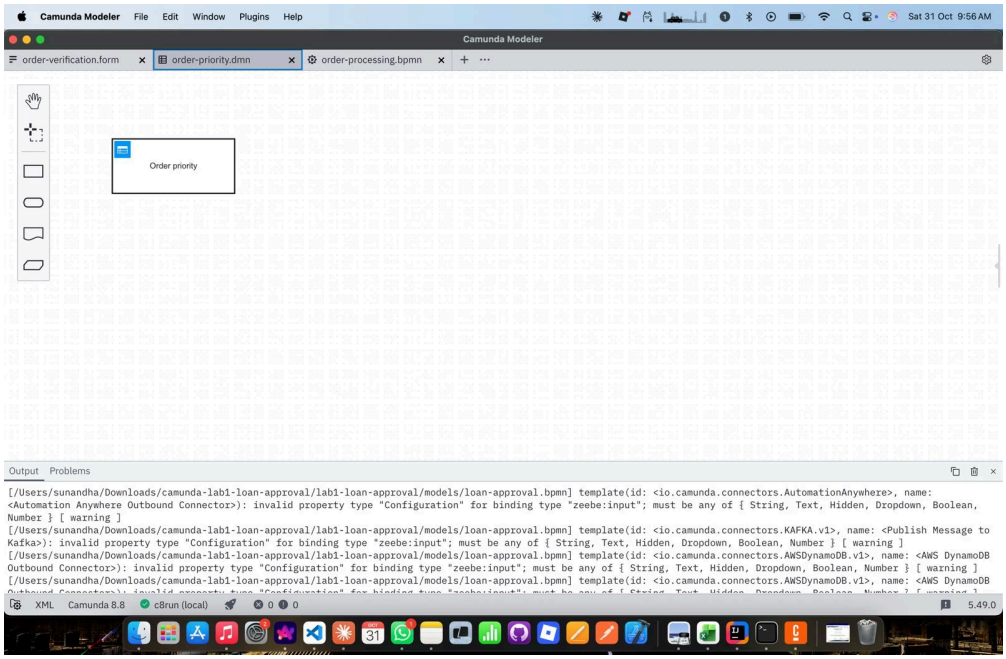


Figure 2: DMN decision requirements diagram (order-priority) in Desktop Modeler

4.3 Spring Boot job worker

`ValidateOrderWorker` (**`@JobWorker(type = "validate-order")`**) performs three duties in one task:

- **Validate the order:** customerRef and paymentToken present, customerCategory REGULAR/GOLD, quantity 1–100, deliveryPreference STANDARD/EXPRESS, known productId. Problems → `orderValid = false` with `validationErrors`.
- **Check simulated inventory:** looks up the product in `InventoryService` and sets `availableStock` and `inStock = availableStock >= quantity`.
- **Calculate the order total:** `orderTotal = unitPrice × quantity` + 15 for EXPRESS delivery.

productId	Product	Price	Stock
P-100	Wireless Mouse	25	50
P-200	Mechanical Keyboard	80	3
P-300	Laptop	1200	10

If the inventory service is switched to "down" (`POST /mock/inventory/outage?down=true`), the lookup throws `IllegalStateException("Inventory service unavailable")`. The job fails, retries are exhausted and Zeebe raises an incident. Three JUnit tests cover the express total, out-of-stock and outage cases (`mvn test`: 3 passed).

4.4 REST Connector and mock payment gateway

Setting	Value
Method / URL / auth	POST `http://localhost:8091/api/payments`, no authentication
Body	`={paymentToken: paymentToken, amount: orderTotal, orderRef: customerRef + "-" + productId}`
Result expression	`={paymentStatus: response.body.status, transactionId: response.body.transactionId, paymentReason: response.body.reason}`
Retries	3, backoff PT5S

Mock behaviour: **`tok_declined`** (or any value not starting with **`tok_`**) returns DECLINED; any other **`tok_...`** returns APPROVED with a transaction id. No real payment credentials are used.

`POST /api/payments {"paymentToken":"tok_ok_visa","amount":50.0,"orderRef":"C-100-P-100"}`

`200 {"status":"APPROVED","transactionId":"TX-ec324684","amount":50.0}`

`POST /api/payments {"paymentToken":"tok_declined","amount":100.0,"orderRef":"C-300-P-100"}`

`200 {"status":"DECLINED","reason":"Card declined by mock gateway","amount":100.0}`



Figure 3: Mock payment gateway called from the terminal

4.5 Manual verification (Tasklist)

Orders with **`orderTotal >= 5000`** are verified by a person. The linked form **`order-verification`** shows customer, product, quantity, total, delivery preference and priority, and collects **`verificationDecision`** (Approve / Reject, required) and an optional **`verificationComment`**.

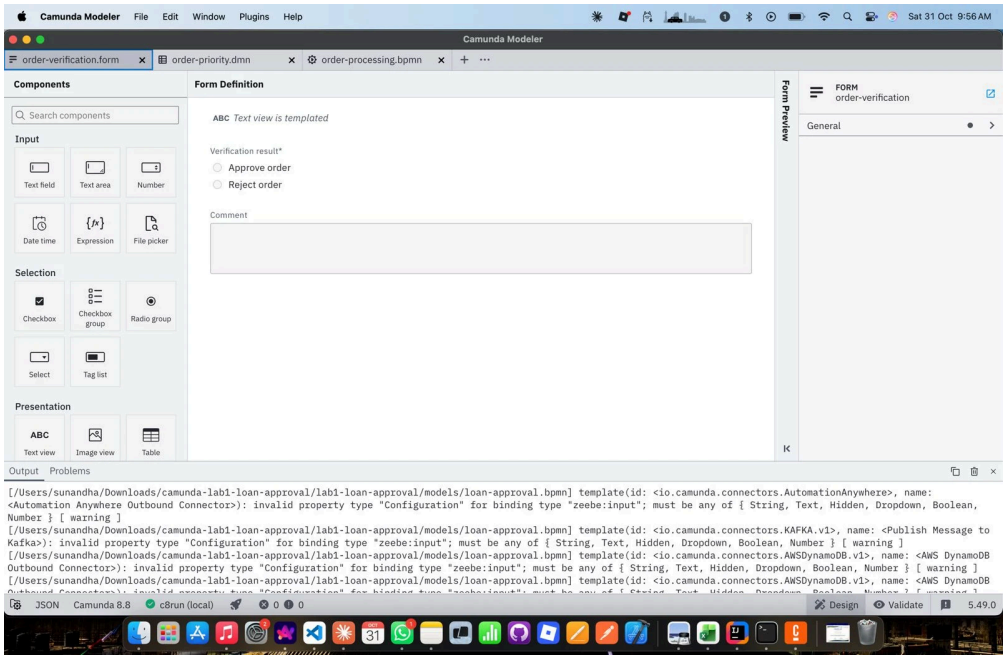


Figure 4: Verification form (order-verification) in Desktop Modeler

4.6 Process-variable dictionary and gateway routing

Variable	Type	Set by	Purpose
customerRef	String	Start	Synthetic customer reference
customerCategory	String	Start	REGULAR / GOLD; DMN input
productId	String	Start	Product ordered
quantity	Number	Start	Units ordered
deliveryPreference	String	Start	STANDARD / EXPRESS; DMN input
paymentToken	String	Start	Mock payment token (no card data)
orderValid / validationErrors	Boolean / List	Worker	Validation result
productName / unitPrice	String / Number	Worker	From the catalog
availableStock / inStock	Number / Boolean	Worker	Stock check
orderTotal	Number	Worker	Price × quantity + delivery fee
deliveryPriority	String	DMN	HIGH / MEDIUM / LOW
verificationDecision / verificationComment	String	Tasklist form	Reviewer outcome
paymentStatus	String	REST Connector	APPROVED / DECLINED
transactionId / paymentReason	String	REST Connector	Gateway reference / decline reason

Gateway	Condition	Route
Valid and in stock?	`orderValid = false`	Invalid order
”	`orderValid and not(inStock)`	Out of stock (not fulfilled)
”	`orderValid and inStock`	Determine priority
High value?	`orderTotal >= 5000`	Verify high-value order
”	default	Process payment
Verified?	`verificationDecision = "APPROVED" / "REJECTED"`	Process payment / Cancelled after review
Payment approved?	`paymentStatus = "APPROVED"`	Sent to fulfilment
”	default	Payment declined

5. Deployment and Execution

The DMN, the form and the BPMN were deployed from Desktop Modeler to the local cluster (process version 2). The Spring Boot app was started with **`mvn spring-boot:run`**, and each scenario was started through the Orchestration Cluster REST API:

```
curl -u demo:demo -H "Content-Type: application/json" \

  -X POST http://localhost:8080/v2/process-instances -d @test-data/1-fulfilled.json
```

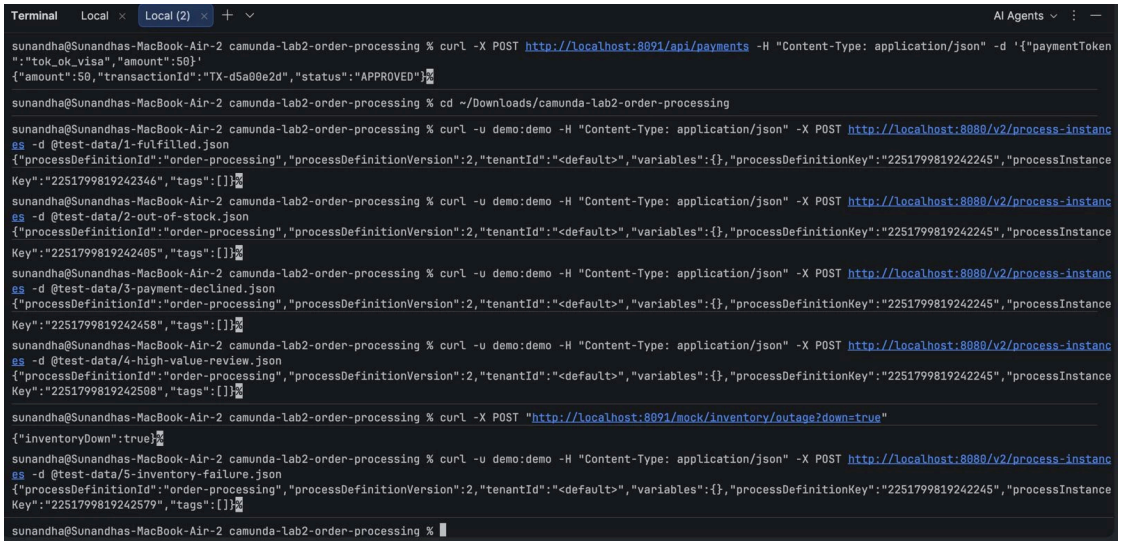


Figure 5: Starting the five scenario instances and switching the inventory outage on

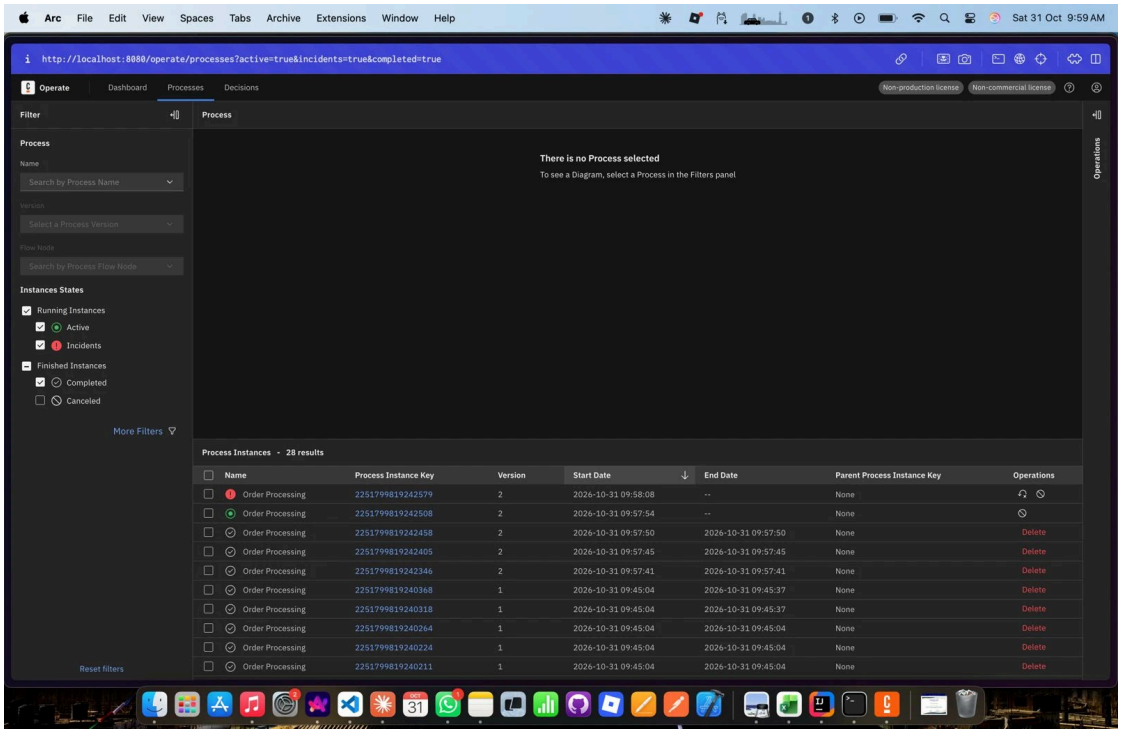


Figure 6: Operate: Order Processing instances (version 2 = test runs for this report)

6. Test Results

All five required scenarios behaved as expected.

#	Scenario	Instance	Actual outcome	Status
1	Stock available, payment succeeds (C-100)	…242346	Total 50, LOW, APPROVED → Sent to fulfilment	Pass
2	Stock insufficient (C-200)	…242405	Qty 5 > stock 3 → Out of stock	Pass
3	Payment declined (C-300)	…242458	Total 100, MEDIUM, DECLINED → Payment declined	Pass
4	High-value manual review (C-400)	…242508	Total 6015, HIGH → Tasklist APPROVED → payment APPROVED → Sent to fulfilment	Pass
5	Inventory failure + recovery (C-500)	…242579	Incident → inventory restored → Retry → Sent to fulfilment	Pass

6.1 Scenario 1: Stock available and payment succeeds

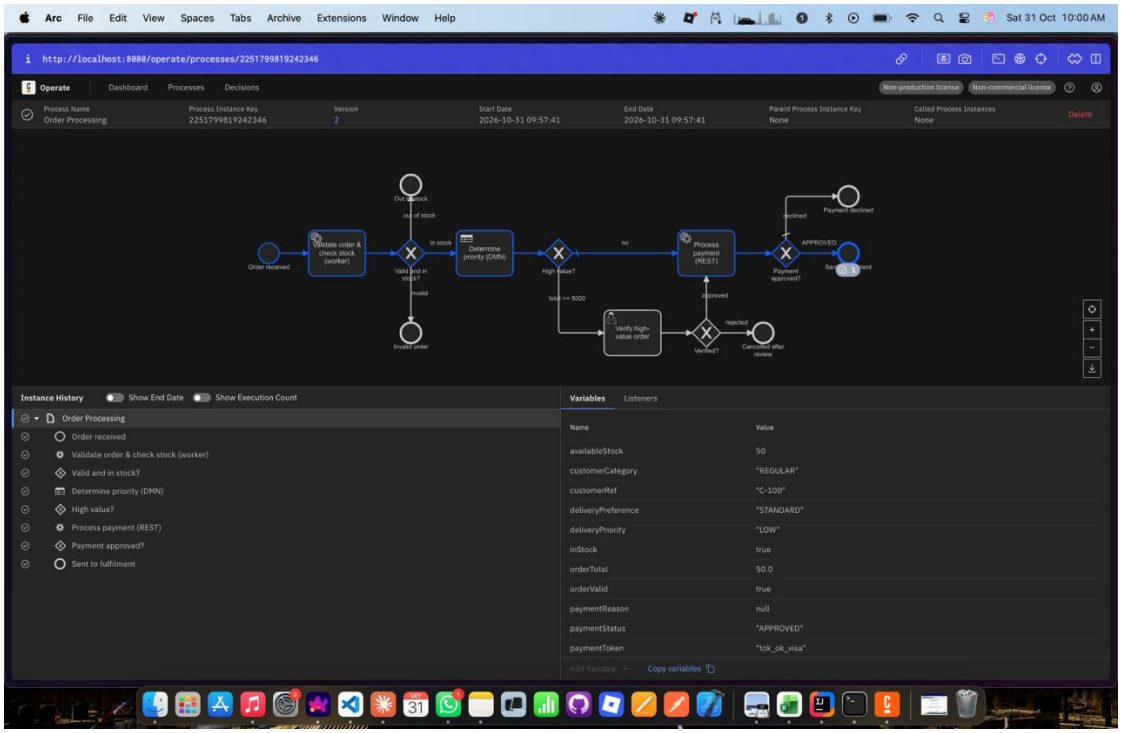


Figure 7: C-100: path to Sent to fulfilment; deliveryPriority LOW, paymentStatus APPROVED

6.2 Scenario 2: Stock insufficient

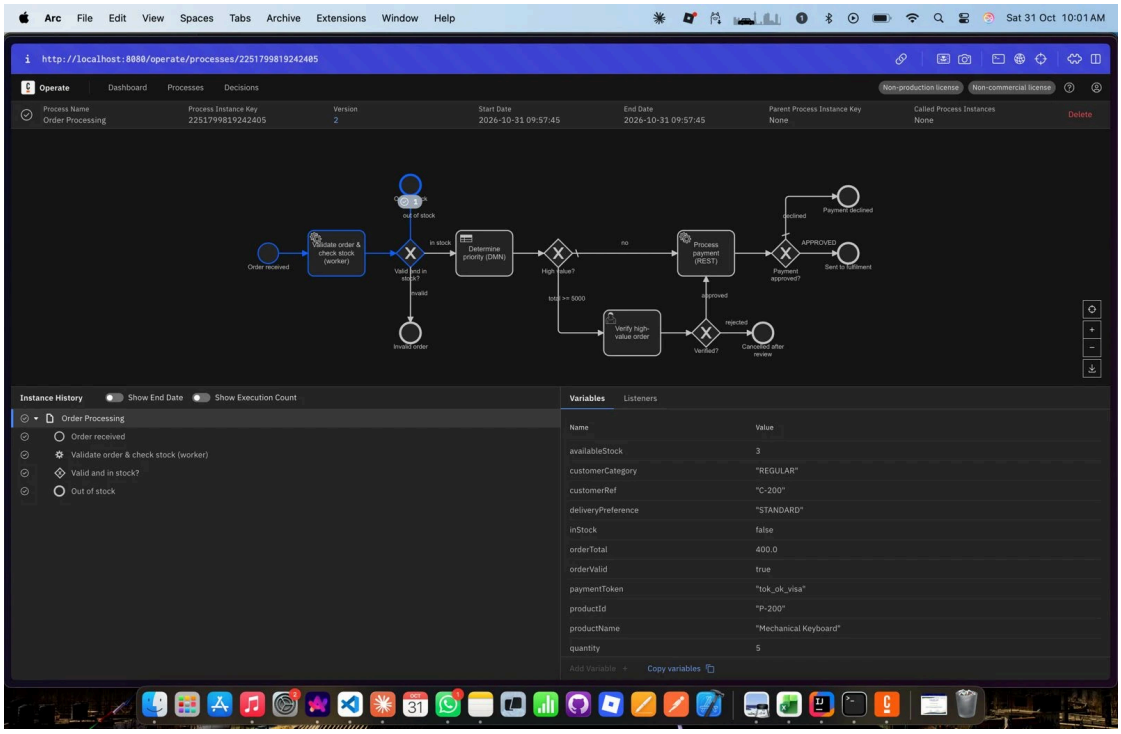


Figure 8: C-200: inStock = false (availableStock 3, quantity 5) → Out of stock

6.3 Scenario 3: Payment declined

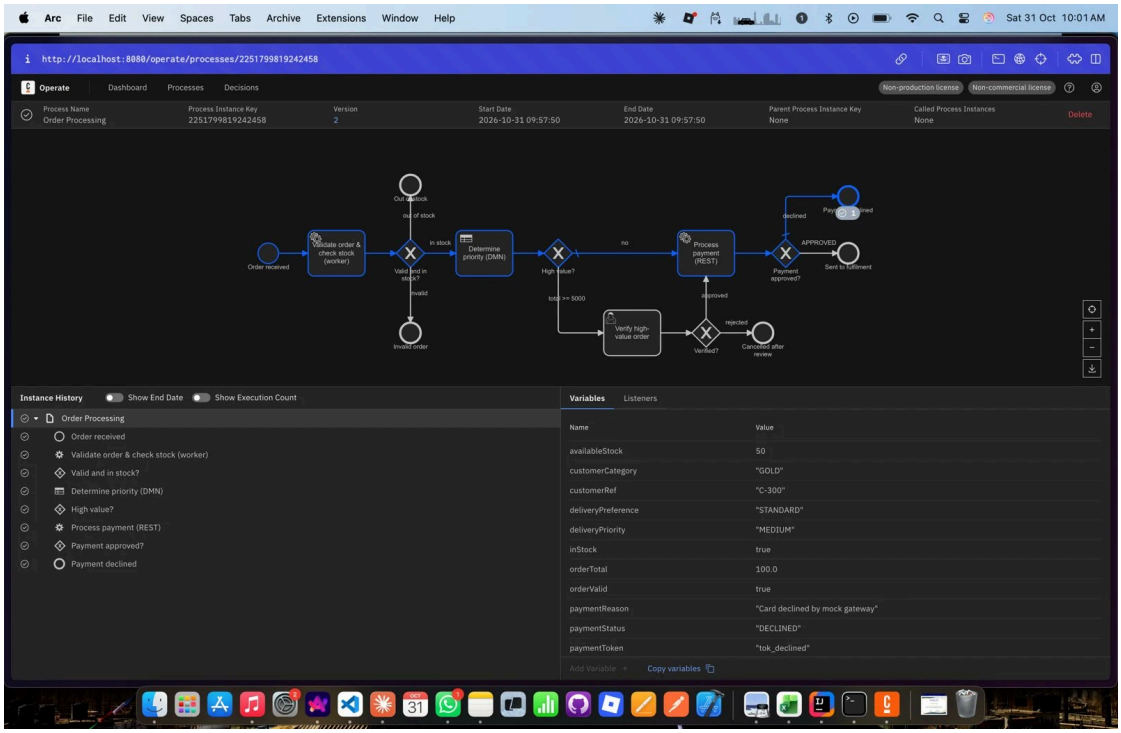


Figure 9: C-300: mock gateway returns DECLINED → Payment declined

6.4 Scenario 4: High-value order routed to manual review

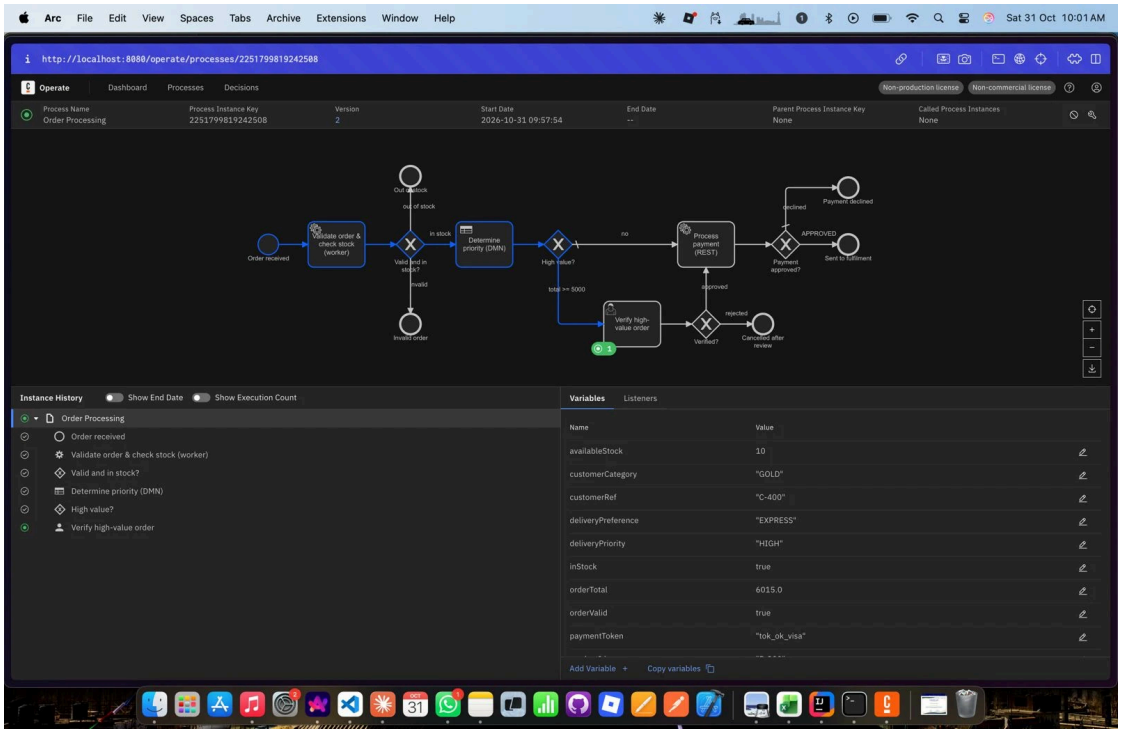


Figure 10: C-400: orderTotal 6015 → waiting at Verify high-value order

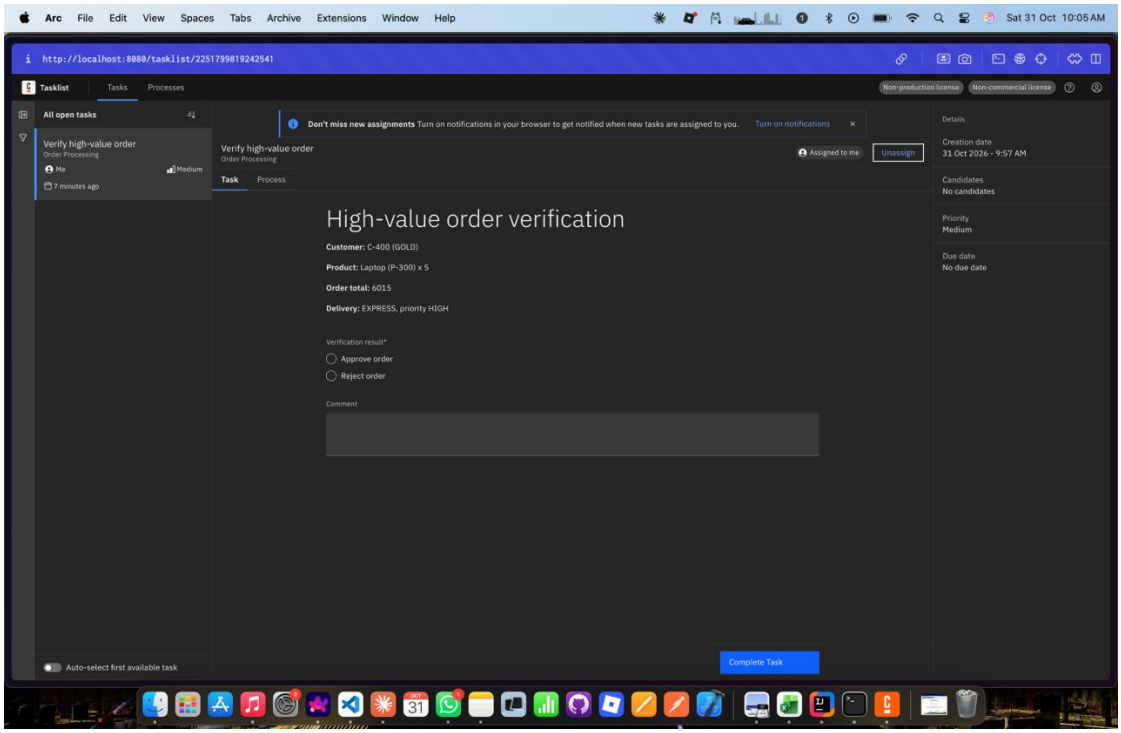


Figure 11: C-400: verification form in Tasklist (Laptop × 5, total 6015, priority HIGH)

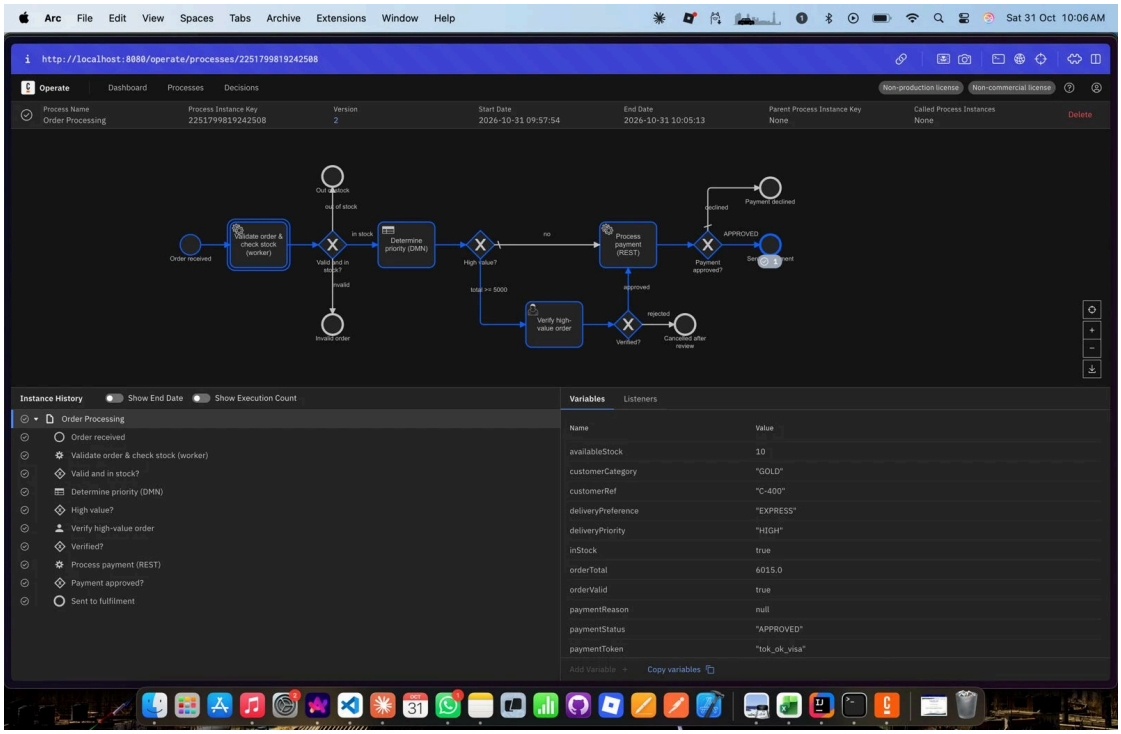


Figure 12: C-400: approved by reviewer → payment APPROVED → Sent to fulfilment

7. Incident Investigation and Recovery

Scenario 5 simulated an inventory-service outage.

1. **Simulate:** `POST /mock/inventory/outage?down=true`, then start order C-500.
2. **Detect:** Operate shows an incident on Validate order & check stock after the worker failed three times (No more retries left).
3. **Diagnose:** the error message is `java.lang.IllegalStateException: Inventory service unavailable - restore it and retry`; no order variables were produced.
4. **Correct:** `POST /mock/inventory/outage?down=false` restored the inventory service.
5. **Retry:** the incident was retried from Operate. The worker succeeded (total 175, priority HIGH), payment was approved and the order reached Sent to fulfilment at 10:07:47.

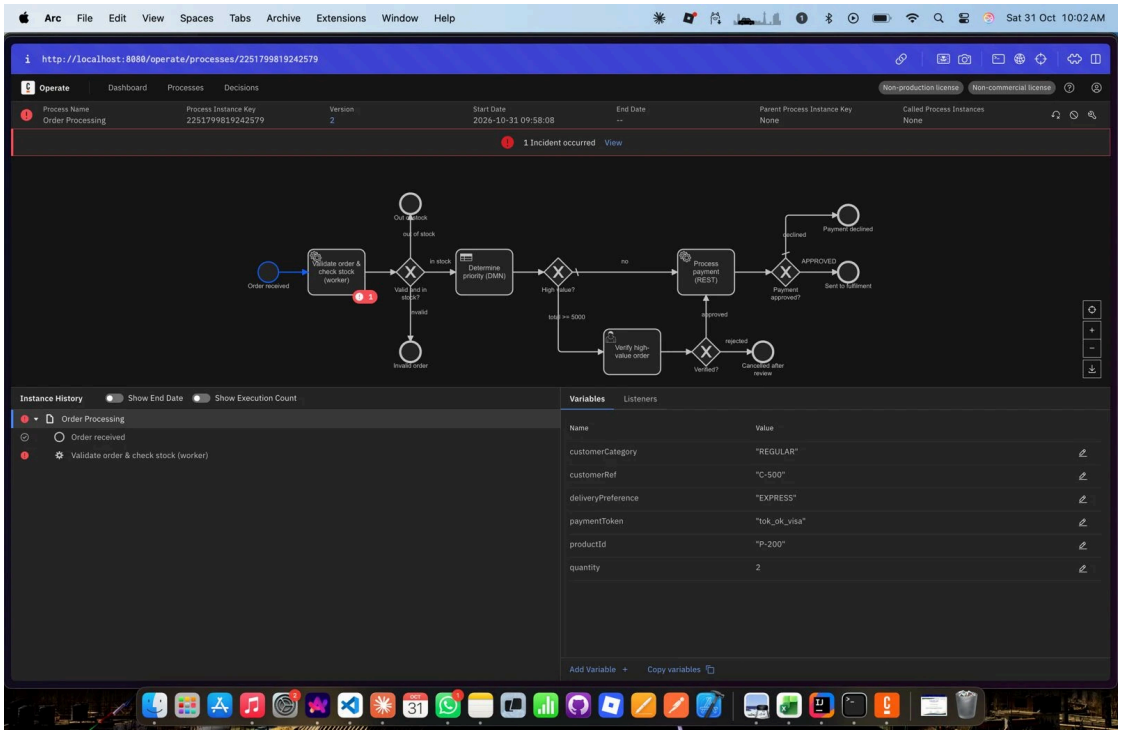


Figure 13: C-500: incident on Validate order & check stock

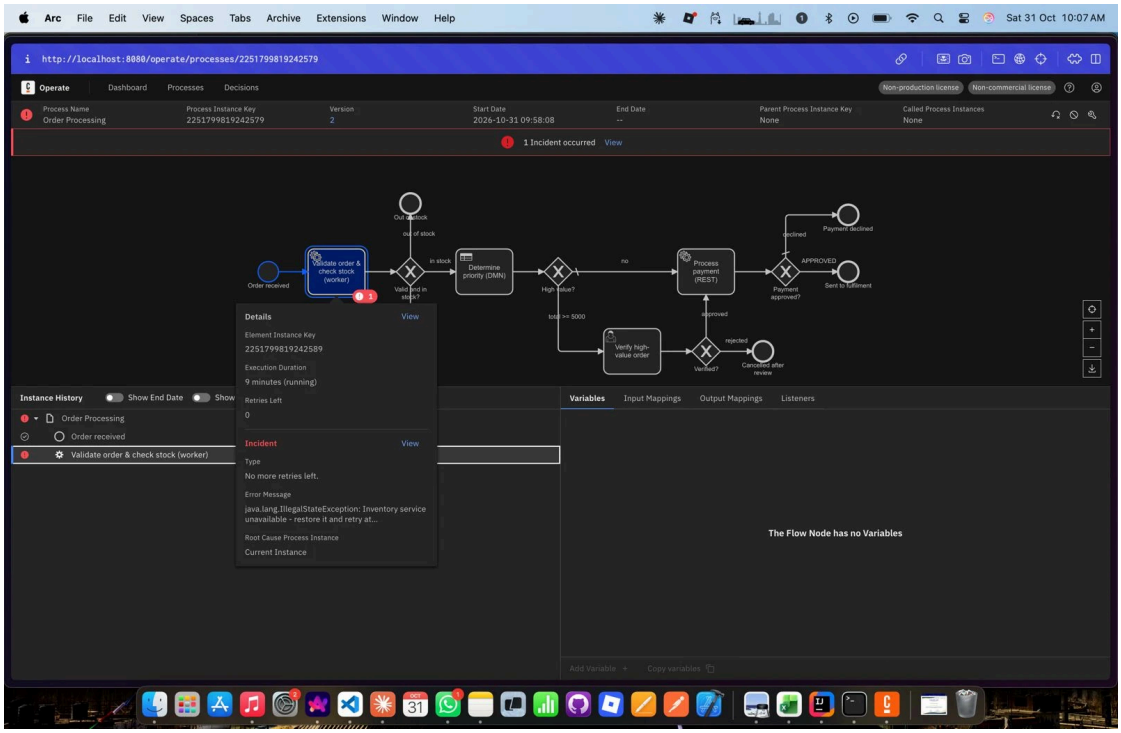


Figure 14: C-500: incident details, Inventory service unavailable, 0 retries left



Figure 15: Corrective action: inventory outage switched off

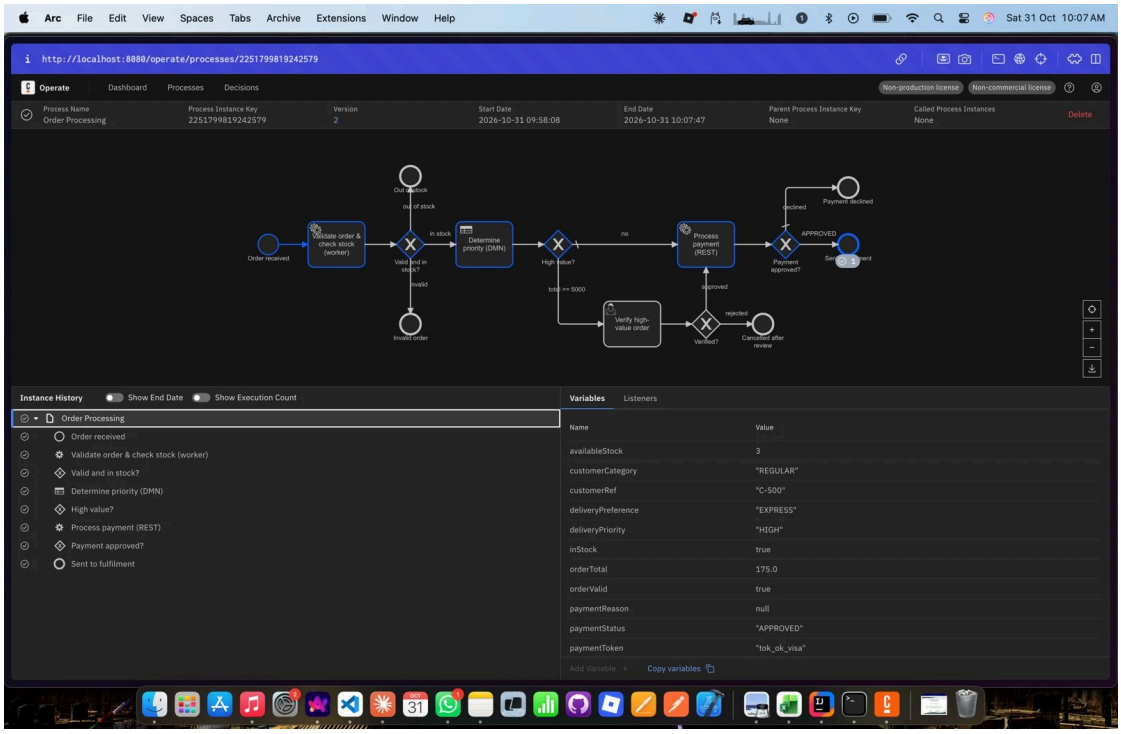


Figure 16: C-500: after Retry the order completes at Sent to fulfilment

8. Assumptions and Limitations

- Catalog, prices and stock are in memory; stock is not decremented after an order.
- The payment gateway is a mock that decides only by token; no real provider or credentials are involved.
- The high-value threshold (5000) and the DMN priority rules are educational assumptions.
- Payment timeouts or 5xx errors are not modelled as a separate path; they retry three times and then raise an incident.
- Verification is assigned to the single `demo` user; everything runs on a local single-node c8run.

9. Conclusion

The order-processing workflow was modelled, deployed and executed end to end on Camunda 8. A Java job worker validated orders, checked inventory and calculated totals; DMN assigned delivery priority; exclusive gateways routed orders on stock availability, order value and payment outcome; a REST Connector integrated the mock payment gateway; and a Tasklist form handled high-value verification. A simulated inventory outage was detected, diagnosed and recovered in Operate. All five required scenarios passed.

Appendix: How to Run

1. Start Camunda: `cd c8run-8.8.32 && ./start.sh`

2. Start the worker and mock services: `mvn spring-boot:run` (port 8091).
3. Deploy `order-priority.dmn`, `order-verification.form` and `order-processing.bpmn` from Desktop Modeler.
4. Start scenarios with the curl command in section 5 using the files in `test-data/`.
5. For scenario 5, switch the outage on before starting and off before retrying.